

Cloud Network & Access Control Architecture for Internal Applications AWS Implementation Report

Prepared by: Thiruppathi R

1. Introduction

This project demonstrates how to design a secure cloud network architecture in AWS for hosting an internal application that must be accessible to employees while remaining protected from external threats.

Previously, systems were directly exposed to the internet without firewall rules, which led to security incidents. The goal of this project is to redesign the network using proper subnet segregation, firewall controls, and access restrictions following cloud security best practices.

2. Objectives

- Design a Virtual Private Cloud (VPC)
 - Create subnet segregation (Public and Private)
 - Assign IP ranges correctly
 - Configure Security Groups and Network ACLs
 - Restrict administrative access (SSH) to authorized user only
 - Allow application access only through HTTP/HTTPS from internal network
 - Simulate insecure configuration and then fix it
 - Deploy a working web application in the private subnet using Docker
-

3. High Level Architecture

- Public Subnet: Represents employee network and bastion access
- Private Subnet: Hosts the internal web application
- NAT Gateway: Allows private subnet outbound internet access
- Internet Gateway: Provides internet access to public subnet

Architecture Flow:

Admin (Laptop) → SSH → Public EC2 (Bastion) → SSH → Private EC2 (Web App)
Employees (Public EC2) → HTTP/HTTPS → Private EC2 (Web App)

4. Network Design

VPC

- CIDR Block: 10.0.0.0/16

Subnets

Subnet	CIDR	Purpose
Public Subnet	10.0.1.0/24	Employee machines and Bastion
Private Subnet	10.0.2.0/24	Web application server

5. Internet Gateway and Route Tables

Public Route Table

Destination	Target
0.0.0.0/0	Internet Gateway

Associated with Public Subnet.

Private Route Table (Before NAT)

Destination	Target
10.0.0.0/16	local

Private subnet had no internet access initially.

6. Simulating the Insecure Design (Before Fix)

A security group was created with the following rule:

Type	Port	Source
All Traffic	All	0.0.0.0/0

This exposed the server completely to the internet and demonstrated the real-world mistake that led to security incidents.

(Screenshot included)

7. Secure Security Group Configuration (After Fix)

Security Group – Public EC2 (Employees/Bastion)

Type	Port	Source
SSH	22	Admin Public IP
All Traffic	All	0.0.0.0/0

Security Group – Private EC2 (Web App)

Type	Port	Source
HTTP	80	10.0.1.0/24
HTTPS	443	10.0.1.0/24
SSH	22	10.0.1.0/24

This ensures:

- Only employees can access the application
- Only admin can SSH through bastion
- No direct internet access to the application server

8. Network ACL Configuration (Defense in Depth)

Private Subnet NACL – Inbound

Rule	Port	Source	Action
100	80	10.0.1.0/24	ALLOW
110	443	10.0.1.0/24	ALLOW
120	22	10.0.1.0/24	ALLOW
130	1024-65535	10.0.1.0/24	ALLOW
*	ALL	0.0.0.0/0	DENY

9. NAT Gateway Implementation

Initially, the private EC2 could not install packages because it had no internet access. To resolve this while maintaining security:

- An Elastic IP was allocated
- NAT Gateway was created in Public Subnet
- Private Route Table updated:

Destination	Target
0.0.0.0/0	NAT Gateway

This allowed outbound internet access without allowing inbound traffic.

10. Screenshots of steps involved:

Create VPC and subnets:

The first screenshot shows the 'Create VPC' page in the AWS Management Console. The 'Resources to create' section has 'VPC only' selected. The 'Name tag - optional' field contains 'my-vpc-01'. The 'IPv4 CIDR block' section has 'IPv4 CIDR manual input' selected, with the CIDR block '10.0.0.0/16' entered. The 'IPv6 CIDR block' section has 'No IPv6 CIDR block' selected. The 'Tenancy' section is set to 'Default'.

The second screenshot shows the 'Create subnet' page. The 'Subnet name' field contains 'public'. The 'Availability Zone' is set to 'No preference'. The 'IPv4 VPC CIDR block' is '10.0.0.0/16'. The 'IPv4 subnet CIDR block' is '10.0.1.0/24'. The 'Tags - optional' section has a tag with key 'Name' and value 'public'.

The screenshot shows the AWS Management Console interface for creating a new subnet. The browser address bar displays the URL: `ap-south-1.console.aws.amazon.com/vpccconsole/home?region=ap-south-1#CreateSubnet:`. The console header includes the AWS logo, a search bar, and navigation links for VPC and Subnets. The main content area is titled 'Create subnet' and contains the following fields:

- Subnet name:** A text input field with the value 'private'.
- Availability Zone:** A dropdown menu with the selected option 'No preference'.
- IPv4 VPC CIDR block:** A dropdown menu with the selected option '10.0.0.0/16'.
- IPv4 subnet CIDR block:** A text input field with the value '10.0.2.0/24'.
- Tags - optional:** A section for adding tags, showing a key-value pair 'Name' with value 'private'.

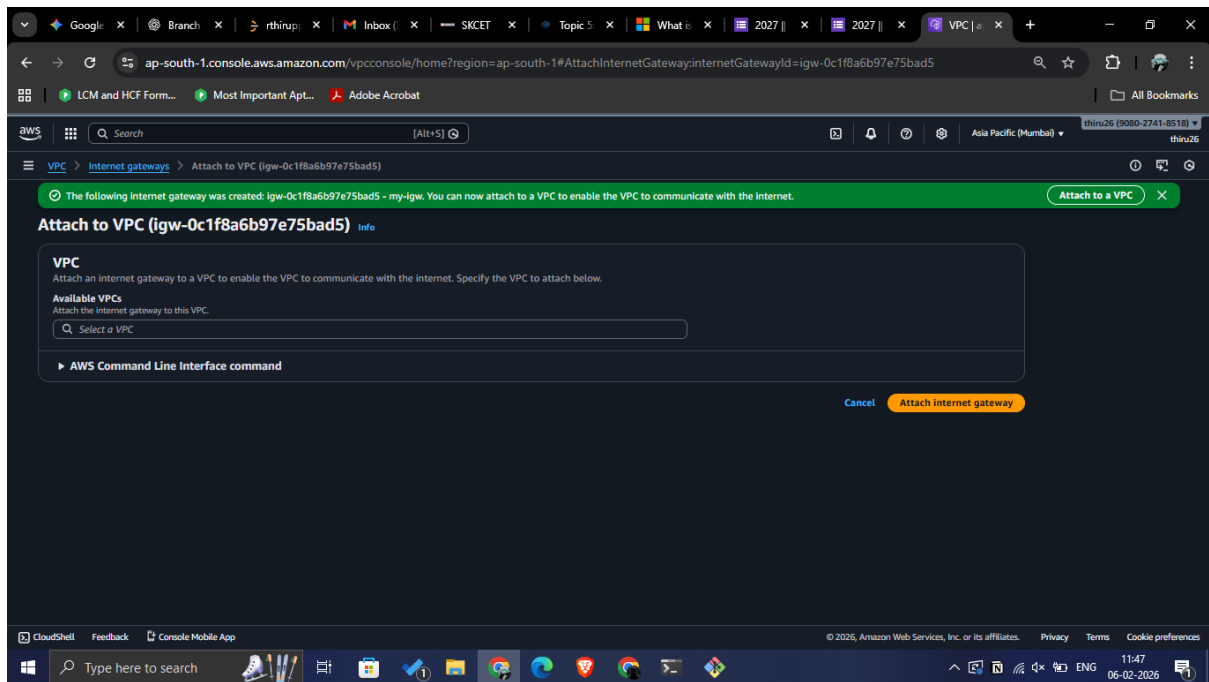
The bottom of the console shows the footer with copyright information and a taskbar with various application icons.

create internet gateway and attach to VPC:

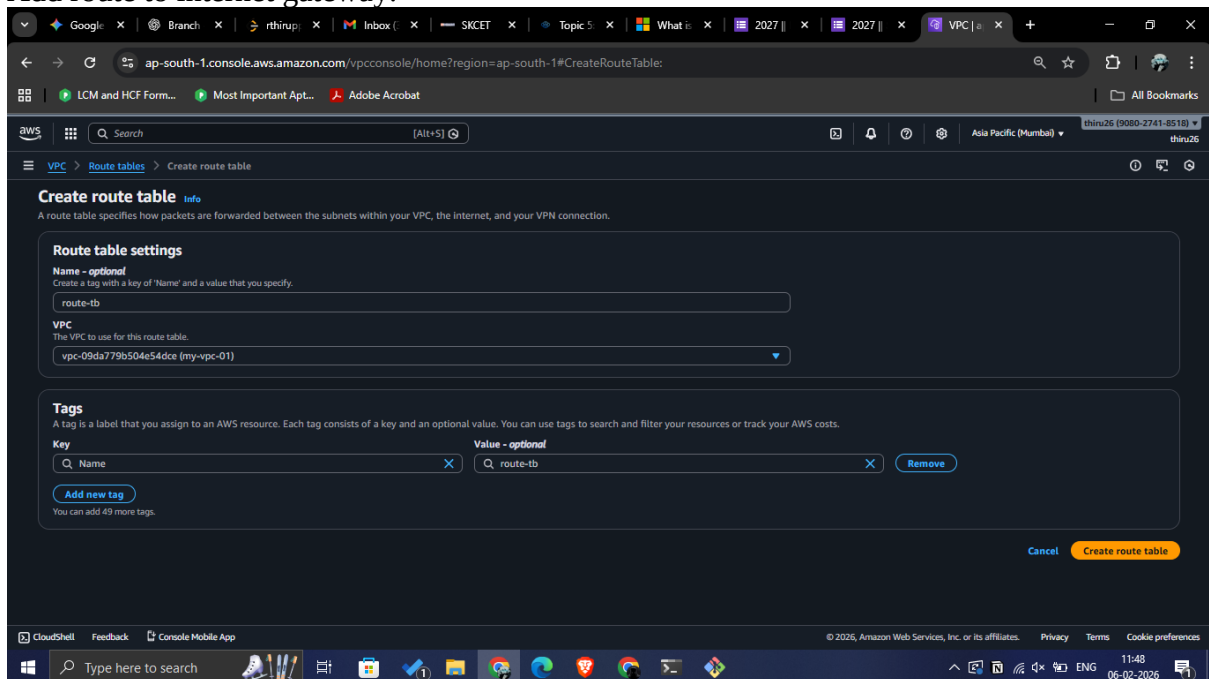
The screenshot shows the AWS Management Console interface for creating a new internet gateway. The browser address bar displays the URL: `ap-south-1.console.aws.amazon.com/vpccconsole/home?region=ap-south-1#CreateInternetGateway:`. The console header includes the AWS logo, a search bar, and navigation links for VPC and Internet gateways. The main content area is titled 'Create internet gateway' and contains the following fields:

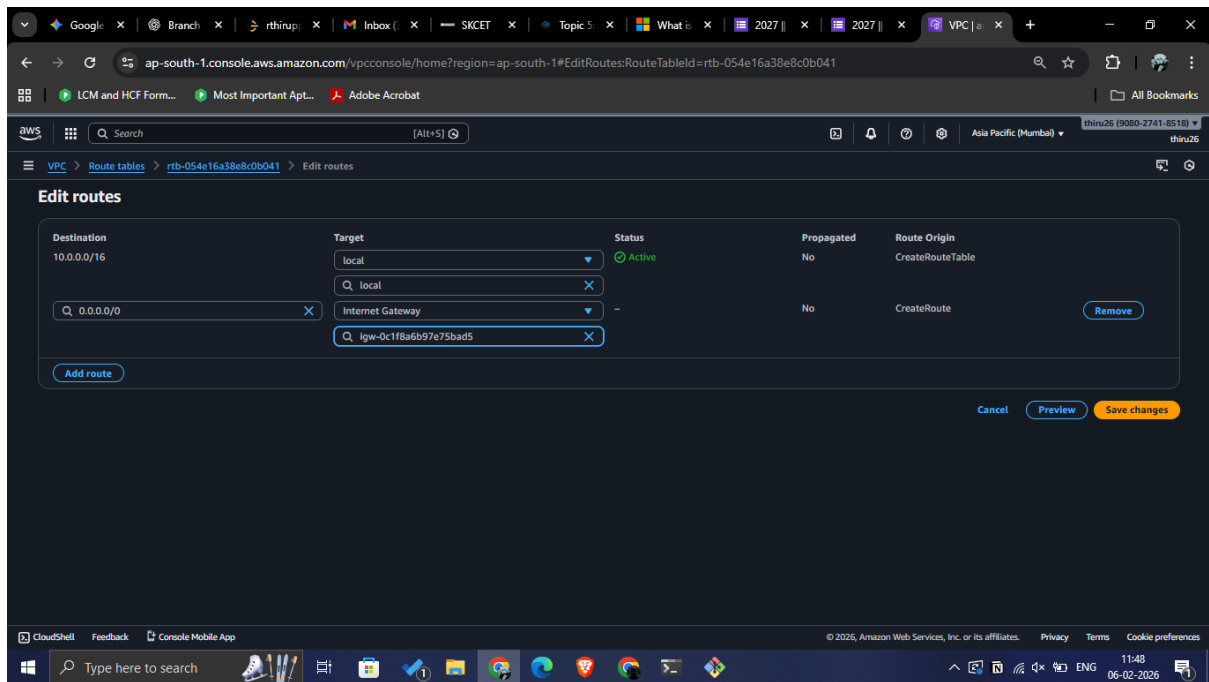
- Internet gateway settings:** A section for configuring the gateway, showing a name tag 'my-igw'.
- Tags - optional:** A section for adding tags, showing a key-value pair 'Name' with value 'my-igw'.

The bottom of the console shows the footer with copyright information and a taskbar with various application icons.

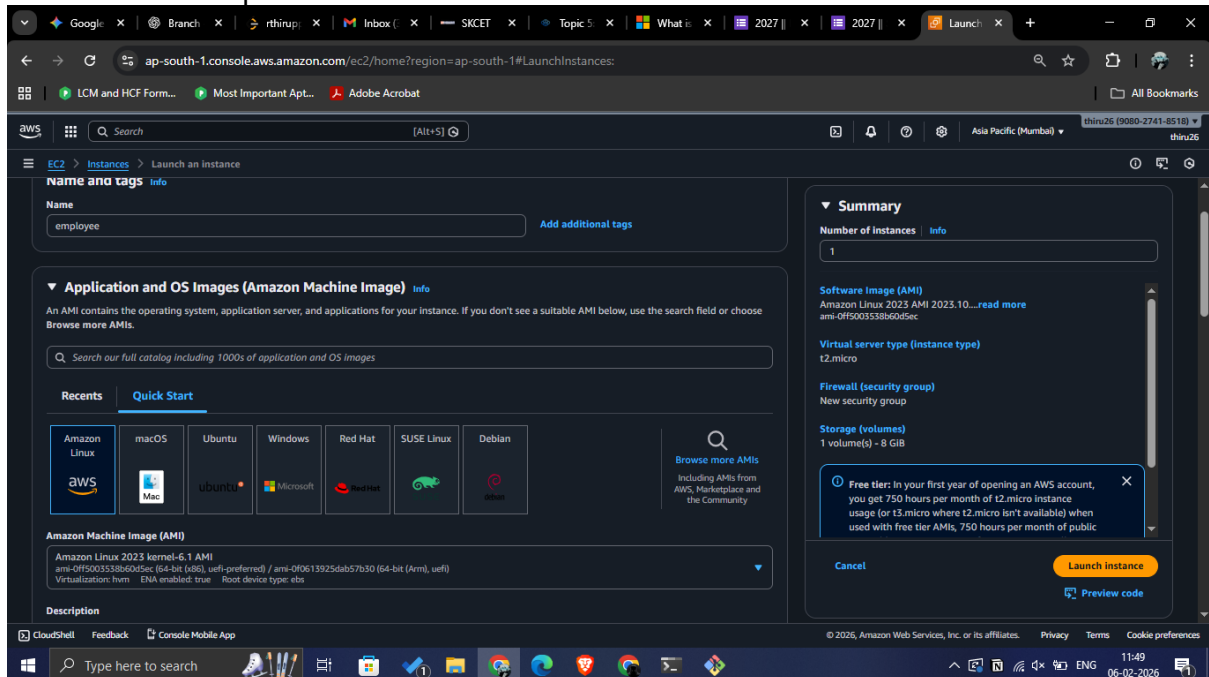


Add route to internet gateway:





Create Public and private EC2:



The image displays two screenshots of the AWS Management Console, specifically the 'Launch Instance' wizard for an Amazon EC2 instance.

Top Screenshot:

- Key pair (login):** The 'Key pair name' field is set to 'cloud-key'. A 'Create new key pair' button is visible.
- Network settings:** The 'Network' dropdown is set to 'vpc-045d284dbcb44675f'. The 'Subnet' dropdown is set to '-'. The 'Auto-assign public IP' option is selected. The 'Firewall (security groups)' section shows 'Create security group' selected, with a note: 'We'll create a new security group called 'launch-wizard-1' with the following rules: Allow SSH traffic from Anywhere (0.0.0.0/0)'. There is also an option to 'Allow HTTPS traffic from the internet'.
- Summary:** Shows 'Number of instances' as 1. The 'Software Image (AMI)' is 'Amazon Linux 2023 AMI 2023.10...'. The 'Virtual server type (instance type)' is 't2.micro'. The 'Firewall (security group)' is 'New security group'. The 'Storage (volumes)' is '1 volume(s) - 8 GiB'. A 'Free tier' notification is present: 'Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public'.

Bottom Screenshot:

- Network settings:** The 'VPC' dropdown is set to 'vpc-09da779b504e54dce (my-vpc-01)'. The 'Subnet' dropdown is set to 'subnet-0abba124fe3afa6e'. The 'Auto-assign public IP' option is set to 'Enable'. The 'Firewall (security groups)' section shows 'Create security group' selected, with a note: 'This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _~:/@#_[]+=&()*\$*'. The 'Security group name' field is set to 'launch-wizard-1'. The 'Description' field is set to 'launch-wizard-1 created 2026-02-06T06:19:15.620Z'.
- Summary:** Shows 'Number of instances' as 1. The 'Software Image (AMI)' is 'Amazon Linux 2023 AMI 2023.10...'. The 'Virtual server type (instance type)' is 't3.micro'. The 'Firewall (security group)' is 'New security group'. The 'Storage (volumes)' is '1 volume(s) - 8 GiB'. A 'Free tier' notification is present: 'Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public'.

The screenshot displays the AWS Management Console interface for launching an EC2 instance. The top navigation bar shows the AWS logo, a search bar, and the region 'Asia Pacific (Mumbai)'. The main content area is titled 'Launch an Instance' and is divided into two main sections: 'Summary' and 'Application and OS Images (Amazon Machine Image)'. The 'Summary' section on the right provides a high-level overview of the configuration, including the number of instances (1), the software image (AMI) 'Amazon Linux 2023.10', the virtual server type (instance type) 't3.micro', the firewall (security group) 'New security group', and the storage (volumes) '1 volume(s) - 8 GiB'. The 'Application and OS Images (Amazon Machine Image)' section on the left allows users to select an AMI from a catalog. It includes a search bar and a grid of AMIs categorized by 'Recents' and 'Quick Start'. The 'Quick Start' category shows AMIs for Amazon Linux, macOS, Ubuntu, Windows, Red Hat, SUSE Linux, and Debian. The 'Name and tags' section at the top left shows the instance name 'web-app-server' and an option to 'Add additional tags'. The bottom of the console shows a Windows taskbar with various application icons and a system clock indicating the time is 11:52 on 06-02-2026.

The screenshot displays the AWS Management Console interface for launching an EC2 instance. The 'Launch Instance' wizard is in progress, with the 'Network settings' tab selected. The configuration includes a VPC (vpc-09da779b504e54dce), a Subnet (subnet-0b4e43dd6e9c9b704), and a Firewall (security group) named 'launch-wizard-2'. The 'Summary' tab on the right provides an overview of the instance configuration, including the Software Image (AMI), Virtual server type (t3.micro), and Storage (volumes). The 'Firewall (security groups)' section shows a new security group named 'launch-wizard-2'.

Network settings

- VPC:** vpc-09da779b504e54dce (my-vpc-01)
- Subnet:** subnet-0b4e43dd6e9c9b704
- Auto-assign public IP:** Disable
- Firewall (security groups):** Create security group (launch-wizard-2)

Summary

- Number of instances:** 1
- Software Image (AMI):** Amazon Linux 2023 AMI 2023.10...
- Virtual server type (instance type):** t3.micro
- Firewall (security group):** New security group
- Storage (volumes):** 1 volume(s) - 8 GiB

Firewall (security groups)

- Security group name:** launch-wizard-2

Security group rule 1 (TCP, 443, 10.0.1.0/24)

- Type:** HTTP
- Source type:** Custom
- Source:** 10.0.1.0/24
- Description - optional:** e.g. SSH for admin desktop

Security group rule 2 (TCP, 22, 10.0.1.0/24)

- Type:** ssh
- Source type:** Custom
- Source:** 10.0.1.0/24
- Description - optional:** e.g. SSH for admin desktop

Summary

- Number of instances:** 1
- Software Image (AMI):** Amazon Linux 2023 AMI 2023.10...
- Virtual server type (instance type):** t3.micro
- Firewall (security group):** New security group
- Storage (volumes):** 1 volume(s) - 8 GiB

The image shows two screenshots of the AWS Management Console. The top screenshot displays the 'Create VPC' page. The 'VPC settings' section is active, showing options for 'Resources to create' (VPC only), 'Name tag' (my-vpc), 'IPv4 CIDR block' (10.0.0.0/16), and 'IPv6 CIDR block' (No IPv6 CIDR block). The bottom screenshot displays the 'Create subnet' page. The 'Subnet settings' section is active, showing options for 'Subnet name' (public), 'Availability Zone' (Asia Pacific (Mumbai) / ap-south-1a), 'IPv4 VPC CIDR block' (10.0.0.0/16), and 'IPv4 subnet CIDR block' (10.0.1.0/24). Both screenshots show the AWS console interface with the 'VPC' breadcrumb and the 'Create' button.

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

Name tag - optional [Info](#)
Creates a tag with a key of 'Name' and a value that you specify.

my-vpc

IPv4 CIDR block [Info](#)
☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR
10.0.0.0/16
CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)
☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Tenancy [Info](#)
Default

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 2

Subnet name [Info](#)
Create a tag with a key of 'Name' and a value that you specify.

public
The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

Asia Pacific (Mumbai) / ap-south-1a

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block
10.0.1.0/24 256 IPs

Tags - optional [Info](#)
Key Value - optional

Name public Remove

The image displays two screenshots of the AWS Management Console interface, showing the process of creating a subnet and launching an EC2 instance.

Top Screenshot: Create subnet

- Page Title:** ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#CreateSubnet:
- Breadcrumbs:** VPC > Subnets > Create subnet
- Subnet 2 of 2:**
 - Subnet name:** Create a tag with a key of 'Name' and a value that you specify. (The name can be up to 256 characters long.)
 - Availability Zone:** Choose the zone in which your subnet will reside, or let Amazon choose one for you.
 - IPv4 VPC CIDR block:** Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.
 - IPv4 subnet CIDR block:** (256 IPs)
 - Tags - optional:**
 - Key: Value - optional:

Bottom Screenshot: Launch an instance

- Page Title:** ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#LaunchInstances:
- Breadcrumbs:** EC2 > Instances > Launch an instance
- Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.**
- Name and tags:**
 - Name: (Add additional tags)
- Application and OS Images (Amazon Machine Image):**
 - An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).
 - Search:
 - Quick Start:**
 - Amazon Linux, macOS, Ubuntu, Windows, Red Hat, SUSE Linux, Debian
 - Amazon Machine Image (AMI):** Amazon Linux 2023 kernel-6.1 AMI (ami-0f5003538b60d5ec (64-bit (x86), uefi-preferred) / ami-0f0613925dab57b3D (64-bit (ARM), uefi))
- Summary:**
 - Number of instances:
 - Software Image (AMI): Amazon Linux 2023 AMI 2023.10...read more (ami-0f5003538b60d5ec)
 - Virtual server type (instance type): t3.micro
 - Firewall (security group): New security group
 - Storage (volumes): 1 volume(s) - 8 GiB
 - Buttons: Cancel, Launch instance, Preview code

The image displays two screenshots of the AWS Management Console, specifically the 'Launch an instance' wizard for an Amazon EC2 instance.

Top Screenshot: Key pair (login) and Network settings

- Key pair (login):** The 'Key pair name' is set to 'cloud'. A 'Create new key pair' button is visible.
- Network settings:**
 - VPC:** 'vpc-0476fd2ca798568 (my-vpc)' is selected.
 - Subnet:** 'subnet-0099558672a6be01' is selected. It is in the 'public' availability zone.
 - Auto-assign public IP:** 'Enable' is selected.
 - Firewall (security group):** 'Create security group' is selected.

Summary (Right Panel):

- Number of instances:** 1
- Software Image (AMI):** Amazon Linux 2023 AMI 2023.10...
- Virtual server type (instance type):** t3.micro
- Firewall (security group):** New security group
- Storage (volumes):** 1 volume(s) - 8 GiB
- Buttons:** Cancel, Launch instance, Preview code

Bottom Screenshot: Name and tags and Application and OS Images

A blue banner at the top of the bottom screenshot reads: "It seems like you may be new to launching instances in EC2. Take a walkthrough to learn about EC2, how to launch instances and about best practices." Buttons for "Take a walkthrough" and "Do not show me this message again" are present.

Launch an instance

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags

Name: web-app-server

Application and OS Images (Amazon Machine Image)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search: Search our full catalog including 1000s of application and OS images

Recents **Quick Start**

Amazon Linux, macOS, Ubuntu, Windows, Red Hat, SUSE Linux, Debian

Summary (Right Panel):

- Number of instances:** 1
- Software Image (AMI):** Amazon Linux 2023 AMI 2023.10...
- Virtual server type (instance type):** t3.micro
- Firewall (security group):** New security group
- Storage (volumes):** 1 volume(s) - 8 GiB
- Buttons:** Cancel, Launch instance, Preview code

Network settings

VPC - required
vpc-0476fd2ca7f98568 (my-vpc)

Subnet
subnet-04760b7badffcc6c6
VPC: vpc-0476fd2ca7f98568 Owner: 205930651799 Availability Zone: ap-south-1a (ap1-az1)
Zone type: Availability Zone IP addresses available: 251 CIDR: 10.0.2.0/24

Auto-assign public IP
Disable

Firewall (security group)
Create security group Select existing security group

Security group name - required
launch-wizard-2

Description - required
launch-wizard-2 created 2026-02-06T06:49:15.619Z

Inbound Security Group Rules

Summary

Number of instances
1

Software image (AMI)
Amazon Linux 2023 AMI 2023.10...read more
ami-0H5003538b60d5ec

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Cancel Launch Instance Preview code

Allow ssh,http,https from public subnet (Organization's subnet):

Inbound Security Group Rules

Type	Protocol	Port range	Source type	Source	Description - optional
ssh	TCP	22	Anywhere	0.0.0.0/0	e.g. SSH for admin desktop
HTTPS	TCP	443	Anywhere	0.0.0.0/0	e.g. SSH for admin desktop
HTTP	TCP	80	Anywhere	0.0.0.0/0	e.g. SSH for admin desktop

Summary

Number of instances
1

Software image (AMI)
Amazon Linux 2023 AMI 2023.10...read more
ami-0H5003538b60d5ec

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Cancel Launch Instance Preview code

The screenshot displays the AWS Management Console interface for an Internet Gateway (igw-00580ba9d06639ec3) in the 'my-igw' state. The console is in the 'ap-south-1' region. A green notification bar at the top states: 'The following internet gateway was created: igw-00580ba9d06639ec3 - my-igw. You can now attach to a VPC to enable the VPC to communicate with the internet.' The left sidebar shows the 'VPC dashboard' with options like 'Your VPCs', 'Subnets', 'Route tables', 'Internet gateways', 'Egress-only internet gateways', 'DHCP option sets', 'Elastic IPs', 'Managed prefix lists', 'NAT gateways', 'Peering connections', 'Route servers', 'Security', and 'PrivateLink and PrivateConnect'. The main content area shows the 'Details' for the Internet Gateway, including its ID, State (Detached), VPC ID, and Owner. Below this, the 'Tags' section shows a single tag with the key 'Name' and value 'my-igw'. The 'Attach to VPC' step is highlighted, showing a search for available VPCs. The search results show 'vpc-0476fdd2ca7f9856d'. The 'Attach internet gateway' button is visible at the bottom right of the 'Attach to VPC' section.

ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#InternetGateway:internetGatewayId=igw-00580ba9d06639ec3

VPC dashboard < AWS Global View Filter by VPC

Virtual private cloud Your VPCs Subnets Route tables Internet gateways Egress-only internet gateways DHCP option sets Elastic IPs Managed prefix lists NAT gateways Peering connections Route servers Security Network ACLs Security groups PrivateLink and PrivateConnect

igw-00580ba9d06639ec3 / my-igw

The following internet gateway was created: igw-00580ba9d06639ec3 - my-igw. You can now attach to a VPC to enable the VPC to communicate with the internet. Attach to a VPC

Details info Internet gateway ID igw-00580ba9d06639ec3 State Detached VPC ID - Owner 205930651799

Tags (1) Search tags Key Value Name my-igw Manage tags

Attach to VPC (igw-00580ba9d06639ec3) info

VPC Attach an internet gateway to a VPC to enable the VPC to communicate with the internet. Specify the VPC to attach below.

Available VPCs Attach the internet gateway to this VPC. vpc-0476fdd2ca7f9856d

AWS Command Line Interface command

Cancel Attach internet gateway

The screenshot displays the AWS VPC console interface for editing routes in the 'ap-south-1' region. The 'Edit routes' page shows a table with two routes. The first route has a destination of '10.0.0.0/16', a target of 'local', and a status of 'Active'. The second route has a destination of '0.0.0.0/0', a target of 'Internet Gateway', and a status of '-'. The 'Propagated' column shows 'No' for both routes, and the 'Route Origin' column shows 'CreateRouteTable' for the first and 'CreateRoute' for the second. A 'Remove' button is visible next to the second route. At the bottom of the console, there are 'Cancel', 'Preview', and 'Save changes' buttons.

Below the console, a terminal window shows the command prompt of a Windows machine. The user attempts to connect to the host '13.232.24.110' using 'ssh -i "cloud.pem" ec2-user@13.232.24.110'. The connection is initially timed out. After a second attempt, the connection is established. The terminal output shows the SSH fingerprint and the user is prompted to confirm the connection. The user responds 'yes', and the terminal displays the 'Amazon Linux 2023' logo and the URL 'https://aws.amazon.com/linux/amazon-linux-2023'. The prompt then changes to '[ec2-user@ip-10-0-1-48 ~]\$'.

The image is a screenshot of a Windows desktop environment. The top half shows a terminal window with a Windows command prompt and an SSH session. The bottom half shows the AWS Management Console for creating a NAT gateway.

Terminal Window:

```
C:\Users\ASUS\Downloads>scp -i cloud.pem cloud.pem ec2-user@13.232.24.110:/home/ec2-user/  
cloud.pem  
100% 1674 34.8KB/s 00:00  
  
C:\Users\ASUS\Downloads>ssh -i "cloud.pem" ec2-user@13.232.24.110  
  
#  
#####  
Amazon Linux 2023  
#####  
#  
#####  
https://aws.amazon.com/linux/amazon-linux-2023  
#####  
#  
#####  
Last login: Fri Feb 6 06:53:03 2026 from 61.1.189.195  
[ec2-user@ip-10-0-1-48 ~]$ chmod 400 cloud-key.pem  
chmod: cannot access 'cloud-key.pem': No such file or directory  
[ec2-user@ip-10-0-1-48 ~]$ chmod 400 cloud.pem  
[ec2-user@ip-10-0-1-48 ~]$ ssh -i cloud.pem ec2-user@10.0.2.92  
The authenticity of host '10.0.2.92 (10.0.2.92)' can't be established.  
ED25519 key fingerprint is SHA256:WvOxSpCkvMAoqWBfAuLDFX6dbLKEQeX18iLnXzkCZgo.  
This key is not known by any other names  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '10.0.2.92' (ED25519) to the list of known hosts.  
  
#  
#####  
Amazon Linux 2023  
#####  
#  
#####  
https://aws.amazon.com/linux/amazon-linux-2023  
#####  
#  
#####  
[ec2-user@ip-10-0-2-92 ~]$
```

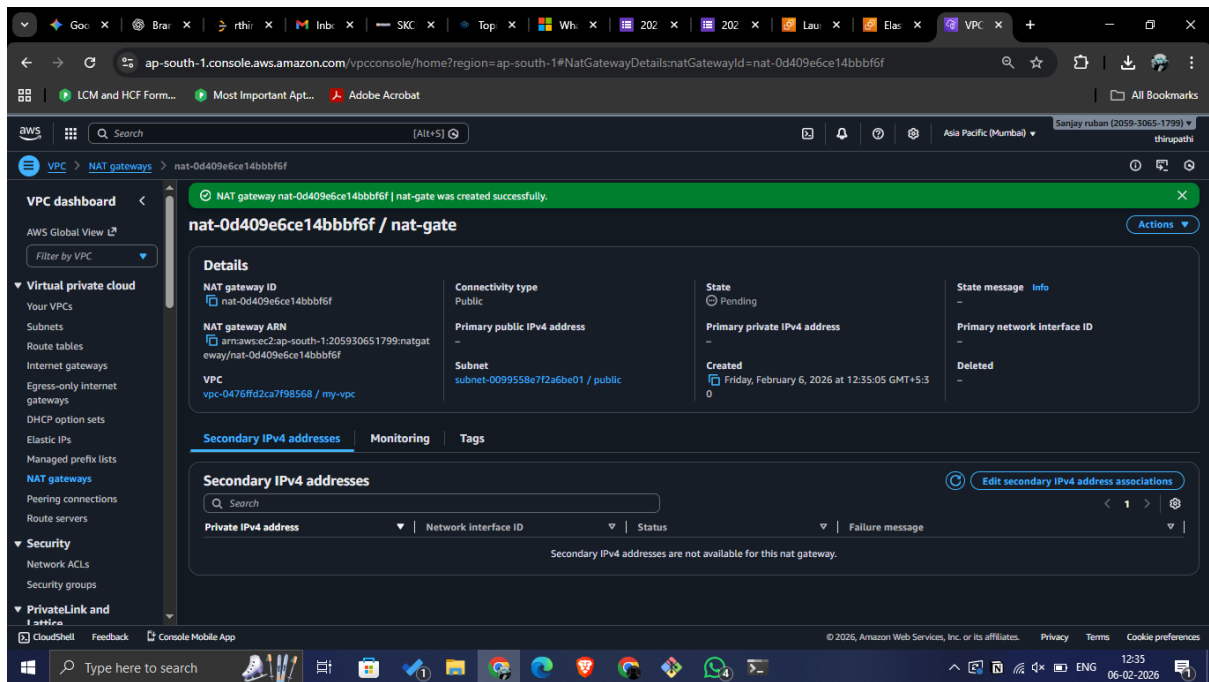
AWS Management Console:

The screenshot shows the "Create NAT gateway" page in the AWS console. The page is titled "Create NAT gateway" and includes a description: "A highly available, managed Network Address Translation (NAT) service that instances in private subnets can use to connect to services in other VPCs, on-premises networks, or the internet."

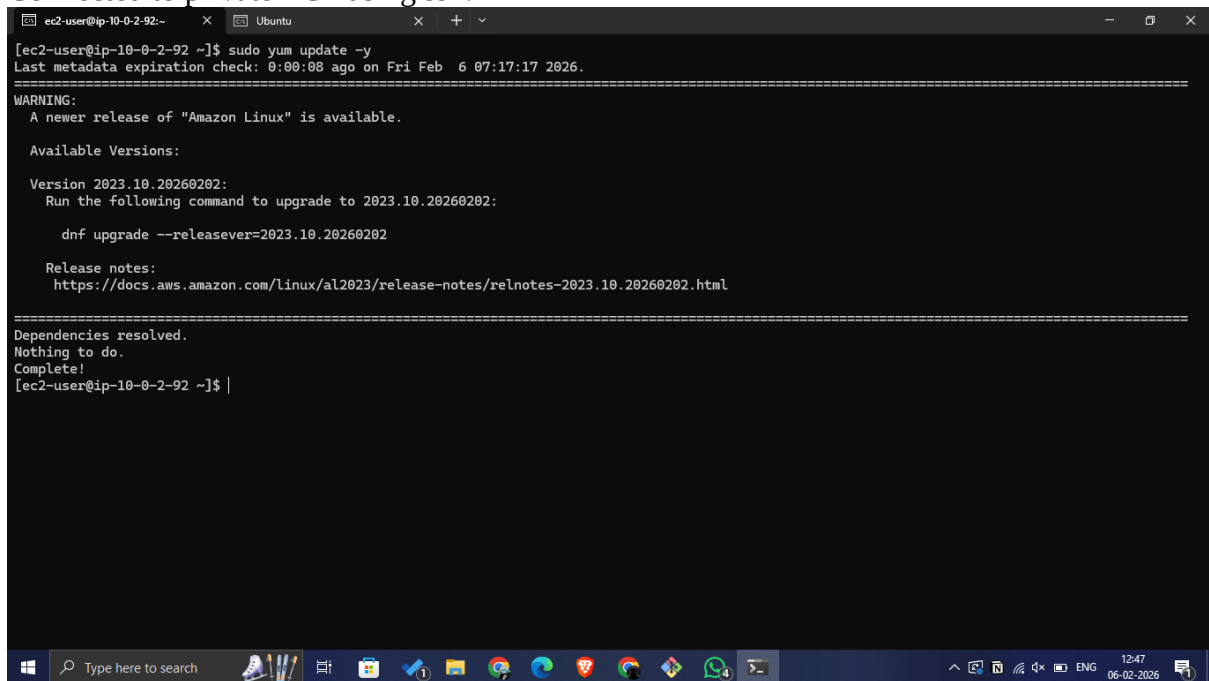
NAT gateway settings

- Name - optional**
Create a tag with a key of 'Name' and a value that you specify.
nat-gate
The name can be up to 256 characters long.
- Availability mode**
Choose whether to deploy across all zones in the region or restrict to a single availability zone.
☐ Regional - new
Scales automatically across all regional AZs, simplifying management for multi AZ deployments.
☒ Zonal
Provides granular control within a specific availability zone, adhering to subnet level settings.
- Subnet**
Select a subnet in which to create the NAT gateway.
subnet-0099558e7f2a6be01 (public)
- Connectivity type**
Select a connectivity type for the NAT gateway.
☒ Public
☐ Private
- Elastic IP allocation ID**
Assign an Elastic IP address to the NAT gateway.
Select on Elastic IP
[Allocate Elastic IP](#)

The bottom of the screenshot shows the Windows taskbar with various application icons and the system clock displaying 12:33 on 06-02-2026.



Connected to private EC2 using ssh:



Update the system and install docker in it:

```
ec2-user@ip-10-0-2-92:~$ sudo yum install docker -y
Complete!
Last metadata expiration check: 0:00:23 ago on Fri Feb  6 07:17:17 2026.
Dependencies resolved.

Package                Architecture      Version           Repository        Size
-----
Installing:
docker                 x86_64            25.0.14-1.amzn2023.0.1  amazonlinux      46 M

Installing dependencies:
container-selinux      noarch            4:2.242.0-1.amzn2023  amazonlinux       58 k
containerd             x86_64            2.1.5-1.amzn2023.0.4  amazonlinux       23 M
iptables-libs          x86_64            1.8.8-3.amzn2023.0.2  amazonlinux       401 k
iptables-nft           x86_64            1.8.8-3.amzn2023.0.2  amazonlinux       183 k
libcgroup              x86_64            3.0-1.amzn2023.0.1    amazonlinux        75 k
libnetfilter_conntrack x86_64            1.0.8-2.amzn2023.0.2  amazonlinux        58 k
libnftnl               x86_64            1.0.1-19.amzn2023.0.2  amazonlinux        30 k
libnftnl               x86_64            1.2.2-2.amzn2023.0.2  amazonlinux        84 k
pigz                   x86_64            2.5-1.amzn2023.0.3    amazonlinux        83 k
runc                   x86_64            1.3.4-1.amzn2023.0.1  amazonlinux        3.9 M

Transaction Summary
-----
Install 11 Packages

Total download size: 74 M
Installed size: 281 M
Downloading Packages:
(1/11): container-selinux-2.242.0-1.amzn2023.noarch.rpm           1.6 MB/s | 58 kB | 00:00
(2/11): iptables-libs-1.8.8-3.amzn2023.0.2.x86_64.rpm           14 MB/s | 401 kB | 00:00
(3/11): iptables-nft-1.8.8-3.amzn2023.0.2.x86_64.rpm            6.8 MB/s | 183 kB | 00:00
(4/11): libcgroup-3.0-1.amzn2023.0.1.x86_64.rpm                 3.9 MB/s | 75 kB | 00:00
(5/11): libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64.rpm  1.9 MB/s | 58 kB | 00:00
(6/11): libnftnl-1.0.1-19.amzn2023.0.2.x86_64.rpm               1.1 MB/s | 30 kB | 00:00

Verifying : docker-25.0.14-1.amzn2023.0.1.x86_64                3/11
Verifying : iptables-libs-1.8.8-3.amzn2023.0.2.x86_64           4/11
Verifying : iptables-nft-1.8.8-3.amzn2023.0.2.x86_64           5/11
Verifying : libcgroup-3.0-1.amzn2023.0.1.x86_64                 6/11
Verifying : libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64  7/11
Verifying : libnftnl-1.0.1-19.amzn2023.0.2.x86_64              8/11
Verifying : libnftnl-1.2.2-2.amzn2023.0.2.x86_64               9/11
Verifying : pigz-2.5-1.amzn2023.0.3.x86_64                     10/11
Verifying : runc-1.3.4-1.amzn2023.0.1.x86_64                   11/11

WARNING:
A newer release of "Amazon Linux" is available.

Available Versions:

Version 2023.10.20260202:
Run the following command to upgrade to 2023.10.20260202:

dnf upgrade --releasever=2023.10.20260202

Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.10.20260202.html

Installed:
container-selinux-4:2.242.0-1.amzn2023.noarch      containerd-2.1.5-1.amzn2023.0.4.x86_64      docker-25.0.14-1.amzn2023.0.1.x86_64
iptables-libs-1.8.8-3.amzn2023.0.2.x86_64        iptables-nft-1.8.8-3.amzn2023.0.2.x86_64    libcgroup-3.0-1.amzn2023.0.1.x86_64
libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64 libnftnl-1.0.1-19.amzn2023.0.2.x86_64      libnftnl-1.2.2-2.amzn2023.0.2.x86_64
pigz-2.5-1.amzn2023.0.3.x86_64                    runc-1.3.4-1.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-10-0-2-92 ~]$ sudo systemctl start docker
[ec2-user@ip-10-0-2-92 ~]$ sudo usermod -aG docker ec2-user
[ec2-user@ip-10-0-2-92 ~]$
```

```
ec2-user@ip-10-0-2-92:~$ sudo yum update
Verifying : libnftnl-1.2.2-2.amzn2023.0.2.x86_64 9/11
Verifying : pigz-2.5-1.amzn2023.0.3.x86_64 10/11
Verifying : runc-1.3.4-1.amzn2023.0.1.x86_64 11/11

WARNING:
A newer release of "Amazon Linux" is available.

Available Versions:

Version 2023.10.20260202:
Run the following command to upgrade to 2023.10.20260202:

dnf upgrade --releasever=2023.10.20260202

Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.10.20260202.html

Installed:
container-selinux-4:2.242.0-1.amzn2023.noarch
iptables-libs-1.8.8-3.amzn2023.0.2.x86_64
libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64
pigz-2.5-1.amzn2023.0.3.x86_64
containerd-2.1.5-1.amzn2023.0.4.x86_64
iptables-nft-1.8.8-3.amzn2023.0.2.x86_64
libnftnl-1.2.2-2.amzn2023.0.2.x86_64
libnftnl-1.2.2-2.amzn2023.0.2.x86_64
docker-25.0.14-1.amzn2023.0.1.x86_64
libcgroup-3.0-1.amzn2023.0.1.x86_64
libnftnl-1.2.2-2.amzn2023.0.2.x86_64
runc-1.3.4-1.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-10-0-2-92 ~]$ sudo systemctl start docker
[ec2-user@ip-10-0-2-92 ~]$ sudo usermod -aG docker ec2-user
[ec2-user@ip-10-0-2-92 ~]$ sudo curl -L "https://github.com/docker/compose/releases/download/v2.20.2/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0  0:00:00  0:00:00  0:00:00     0
 34 57.6M   34 20.0M    0     0  12.3M      0  0:00:04  0:00:01  0:00:03  19.6M
```

Writing docker compose:

```
GNU nano 8.3 docker-compose.yml
version: '3'
services:
  web:
    image: nginx
    ports:
      - "80:80"
```

```
ec2-user@ip-10-0-2-92:~/app X Ubuntu X + v
Run the following command to upgrade to 2023.10.20260202:

dnf upgrade --releasever=2023.10.20260202

Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.10.20260202.html

=====

Installed:
container-selinux-4:2.242.0-1.amzn2023.noarch          containerd-2.1.5-1.amzn2023.0.4.x86_64          docker-25.0.14-1.amzn2023.0.1.x86_64
iptables-libs-1.8.8-3.amzn2023.0.2.x86_64            iptables-nft-1.8.8-3.amzn2023.0.2.x86_64        libcgrouprp-3.0-1.amzn2023.0.1.x86_64
libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64    libnftnl-1.0.1-19.amzn2023.0.2.x86_64          libnftnl-1.2.2-2.amzn2023.0.2.x86_64
pigz-2.5-1.amzn2023.0.3.x86_64                      runc-1.3.4-1.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-10-0-2-92 ~]$ sudo systemctl start docker
[ec2-user@ip-10-0-2-92 ~]$ sudo usermod -aG docker ec2-user
[ec2-user@ip-10-0-2-92 ~]$ sudo curl -L "https://github.com/docker/compose/releases/download/v2.20.2/docker-compose-$(uname -s)-$(uname -m)" -o /usr
/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0:00:00 0:00:00 0:00:00 0
100 57.6M 100 57.6M 0 0 23.9M 0 0:00:02 0:00:02 -- -- 32.1M
[ec2-user@ip-10-0-2-92 ~]$ mkdir app && cd app
[ec2-user@ip-10-0-2-92 app]$ nano docker-compose.yml
[ec2-user@ip-10-0-2-92 app]$ docker-compose up -d
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/c
ontainers/json?all=1&filters=%7B%22label%22%3A%7B%22com.docker.compose.config-hash%22%3Atrue%2C%22com.docker.compose.project%3Dapp%22%3Atrue%7D%7D":
dial unix /var/run/docker.sock: connect: permission denied
[ec2-user@ip-10-0-2-92 app]$ sudo docker-compose up -d
[+] Running 0/1
! web Pulling 3.0s
```

Execute Docker compose up

```
ec2-user@ip-10-0-2-92:~/app X Ubuntu X + v
=====

Installed:
container-selinux-4:2.242.0-1.amzn2023.noarch          containerd-2.1.5-1.amzn2023.0.4.x86_64          docker-25.0.14-1.amzn2023.0.1.x86_64
iptables-libs-1.8.8-3.amzn2023.0.2.x86_64            iptables-nft-1.8.8-3.amzn2023.0.2.x86_64        libcgrouprp-3.0-1.amzn2023.0.1.x86_64
libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64    libnftnl-1.0.1-19.amzn2023.0.2.x86_64          libnftnl-1.2.2-2.amzn2023.0.2.x86_64
pigz-2.5-1.amzn2023.0.3.x86_64                      runc-1.3.4-1.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-10-0-2-92 ~]$ sudo systemctl start docker
[ec2-user@ip-10-0-2-92 ~]$ sudo usermod -aG docker ec2-user
[ec2-user@ip-10-0-2-92 ~]$ sudo curl -L "https://github.com/docker/compose/releases/download/v2.20.2/docker-compose-$(uname -s)-$(uname -m)" -o /usr
/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0:00:00 0:00:00 0:00:00 0
100 57.6M 100 57.6M 0 0 23.9M 0 0:00:02 0:00:02 -- -- 32.1M
[ec2-user@ip-10-0-2-92 ~]$ mkdir app && cd app
[ec2-user@ip-10-0-2-92 app]$ nano docker-compose.yml
[ec2-user@ip-10-0-2-92 app]$ docker-compose up -d
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/c
ontainers/json?all=1&filters=%7B%22label%22%3A%7B%22com.docker.compose.config-hash%22%3Atrue%2C%22com.docker.compose.project%3Dapp%22%3Atrue%7D%7D":
dial unix /var/run/docker.sock: connect: permission denied
[ec2-user@ip-10-0-2-92 app]$ sudo docker-compose up -d
[+] Running 6/8
! web 7 layers [#####] 29.16MB/29.78MB Pulling 6.0s
! 0c8d55a45c0d Extracting [=====] ] 29.16MB/29.78MB 3.0s
! 46bf3a120c8e Download complete 1.3s
! 4f4efe02d542 Download complete 0.9s
! 7b6cb8ccac7b Download complete 1.8s
! f73408a233fd Download complete 2.2s
! 47cd406a84ef Download complete 2.6s
! bae5a1799a80 Download complete 2.7s
```

```

ec2-user@ip-10-0-2-92:~$ cd /usr/local/bin/docker-compose
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %    0      0     0    0         0      0         0      0
100 57.6M 100 57.6M    0     0  23.9M    0:00:02    0:00:02    0:00:02 32.1M

[ec2-user@ip-10-0-2-92 ~]$ mkdir app && cd app
[ec2-user@ip-10-0-2-92 app]$ nano docker-compose.yml
[ec2-user@ip-10-0-2-92 app]$ docker-compose up -d
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/c
ontainers/json?all=1&filters=%7B%22label%22%3A%7B%22com.docker.compose.config-hash%22%3Atrue%2C%22com.docker.compose.project%3Dapp%22%3Atrue%7D%7D":
dial unix /var/run/docker.sock: connect: permission denied
[ec2-user@ip-10-0-2-92 app]$ sudo docker-compose up -d
[+] Running 8/8
✔ web 7 layers [#####] 0B/0B Pulled 7.4s
✔ 0c8d55a45c0d Pull complete 3.1s
✔ 46bf3a120c8e Pull complete 4.3s
✔ 4f4efe02d542 Pull complete 4.3s
✔ 7b6cb8ccac7b Pull complete 4.3s
✔ f73400a233fd Pull complete 4.4s
✔ 47cd406a84ef Pull complete 4.4s
✔ bae5a1799a80 Pull complete 4.4s
[+] Running 2/2
✔ Network app_default Created 0.1s
✔ Container app-web-1 Started 0.7s
[ec2-user@ip-10-0-2-92 app]$ sudo docker-compose ps
NAME                IMAGE                COMMAND                  SERVICE    CREATED        STATUS        PORTS
app-web-1           nginx               "/docker-entrypoint..." web         34 seconds ago Up 33 seconds 0.0.0.0:80->80/tcp, :::80->80/tcp
[ec2-user@ip-10-0-2-92 app]$ docker ps
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.44/c
ontainers/json?": dial unix /var/run/docker.sock: connect: permission denied
[ec2-user@ip-10-0-2-92 app]$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
838aaefaf9c   nginx    "/docker-entrypoint..." 47 seconds ago Up 46 seconds 0.0.0.0:80->80/tcp, :::80->80/tcp app-web-1
[ec2-user@ip-10-0-2-92 app]$ |

```

We can access the ec2 from our employees pc:

```

[ec2-user@ip-10-0-2-92:~] X ec2-user@ip-10-0-1-48:~ X
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Fri Feb  6 07:09:21 2026 from 61.1.189.195
[ec2-user@ip-10-0-1-48 ~]$ curl http://10.0.2.92
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
[ec2-user@ip-10-0-1-48 ~]$ |

```

11. Web Application Deployment (Docker)

Docker and Docker Compose were installed on the Private EC2.

A simple web application using Nginx was deployed using:

docker-compose.yml

- Nginx container exposed on port 80

Employees from Public EC2 successfully accessed the app using:

curl <http://10.0.2.92>

Direct access from internet was not possible.

12. Testing Results

Test	Result
Laptop → Private EC2	Blocked
Laptop → Public EC2	Allowed (SSH)
Public EC2 → Private EC2 (HTTP)	Allowed
Public EC2 → Private EC2 (SSH)	Allowed
Internet → Private EC2	Blocked

13. Security Best Practices Demonstrated

- Principle of Least Privilege
 - Network Segmentation
 - Bastion Host Access
 - Defense in Depth (SG + NACL)
 - Private Subnet Isolation
 - Secure Administrative Access
 - Controlled Application Ports
-

14. Conclusion

This project successfully redesigned an insecure cloud environment into a secure, segmented AWS architecture where:

- Internal applications are protected from internet exposure
- Employees can access the application safely
- Administrative access is tightly controlled
- Best practices in VPC design, routing, firewall rules, and subnet segregation are implemented

This architecture closely resembles real-world enterprise cloud security design.

15. Screenshots Attached

1. VPC and Subnets
2. Route Tables
3. Internet Gateway and NAT Gateway
4. Insecure Security Group (Before)
5. Secure Security Groups (After)
6. Network ACL rules
7. EC2 instances
8. Docker running web app
9. curl test from Public EC2

End of Report

[Thiruppathi R]